

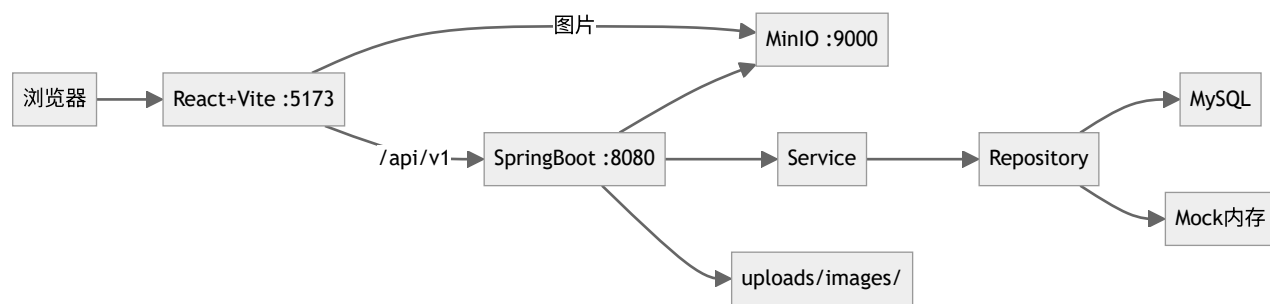
WHU Circle 系统设计说明书

武汉大学 · 校园社交平台原型系统

1. 系统体系架构

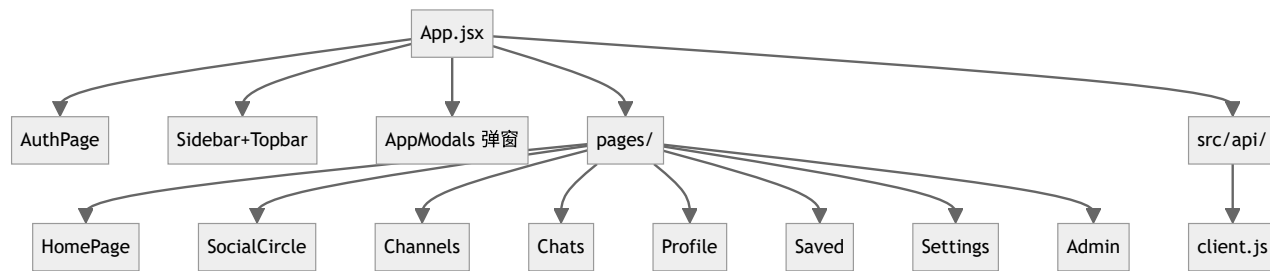
1.1 总体架构

前后端分离四层结构：浏览器 → React + Vite 前端 → Spring Boot 后端 → MySQL（业务数据）+ MinIO（图片文件）。团队联调通过 Tailscale 内网穿透共享组长电脑的 MySQL 和 MinIO。



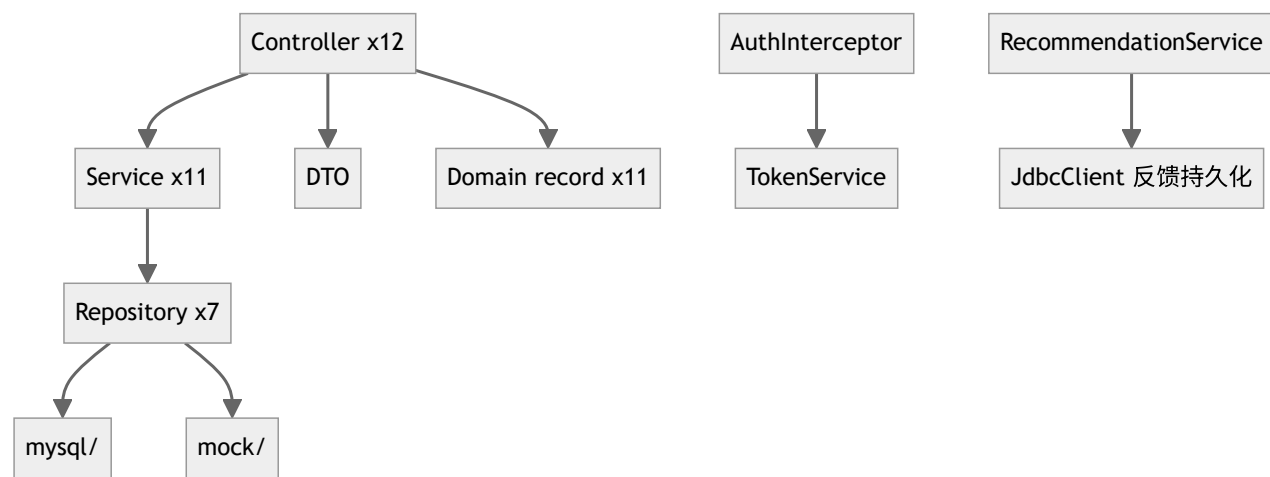
1.2 前端架构

单页应用，App.jsx 集中管理登录态、导航和弹窗状态。src/api/client.js 统一 Bearer Token 与错误处理。

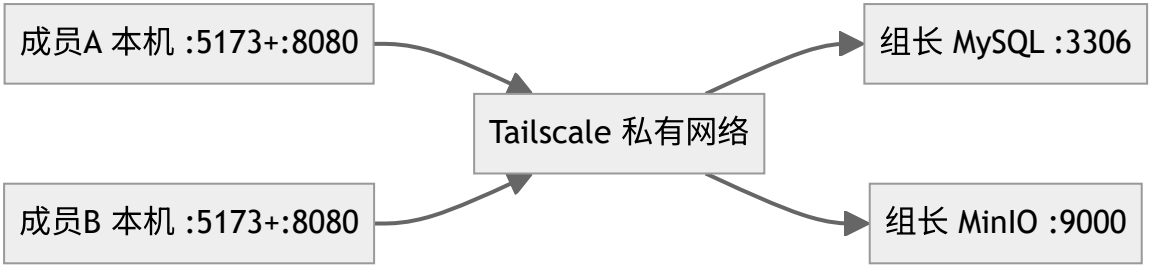


1.3 后端架构

Controller → Service → Repository 三层分层，AuthenticationInterceptor 拦截认证。12 个 Controller、11 个 Service、7 个 Repository 接口，MySQL 和 Mock 双实现。

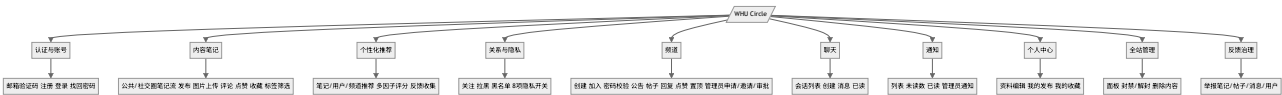


1.4 部署架构



2. 系统功能结构

2.1 功能层次

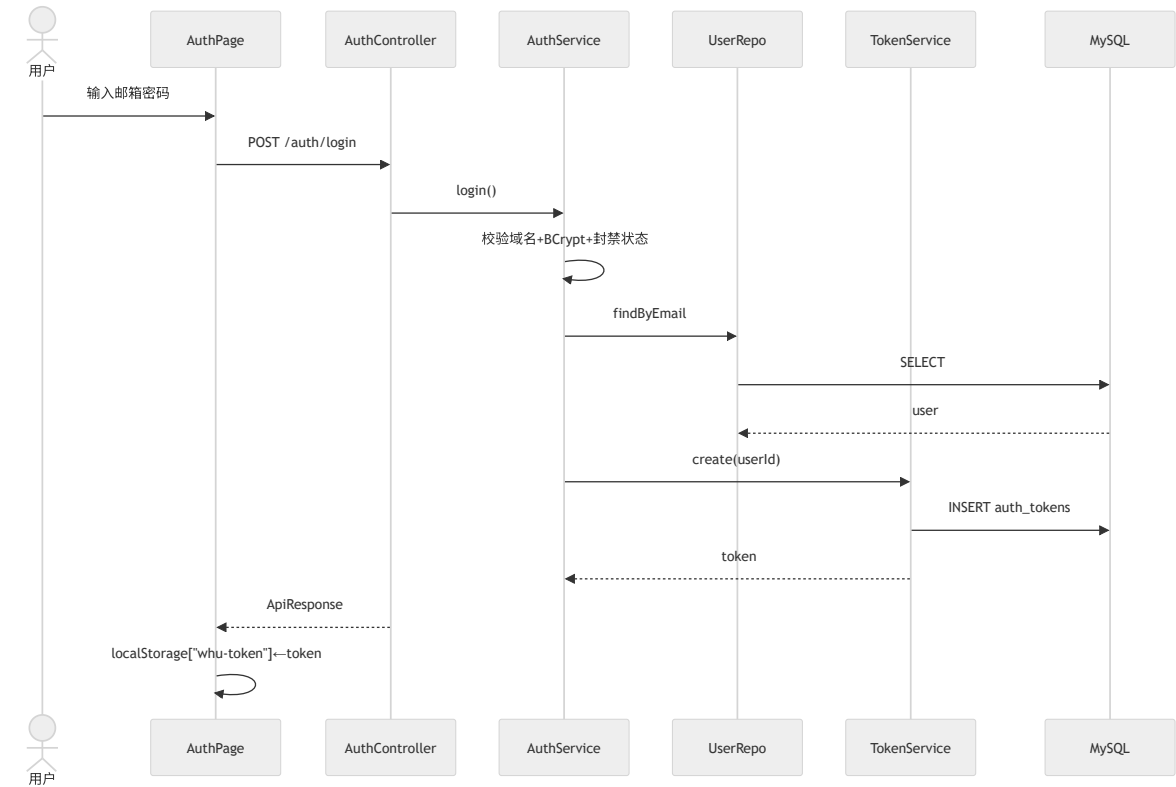


2.2 角色权限

角色	说明
游客	仅可访问登录、注册、找回密码
普通用户	笔记、社交、频道、聊天、通知、设置、推荐等全功能
频道管理员	修改公告、置顶帖子；初始管理员可邀请/审批其他管理员
全站管理员 <code>role=ADMIN</code>	管理面板、封禁用户/频道、删除任意内容

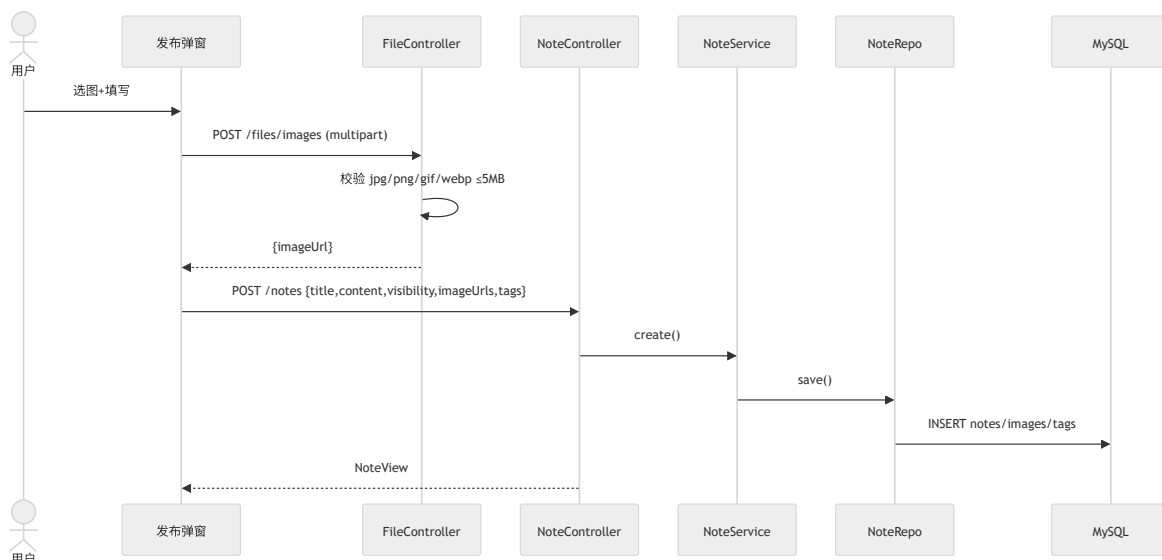
3. 系统用例时序图

3.1 登录



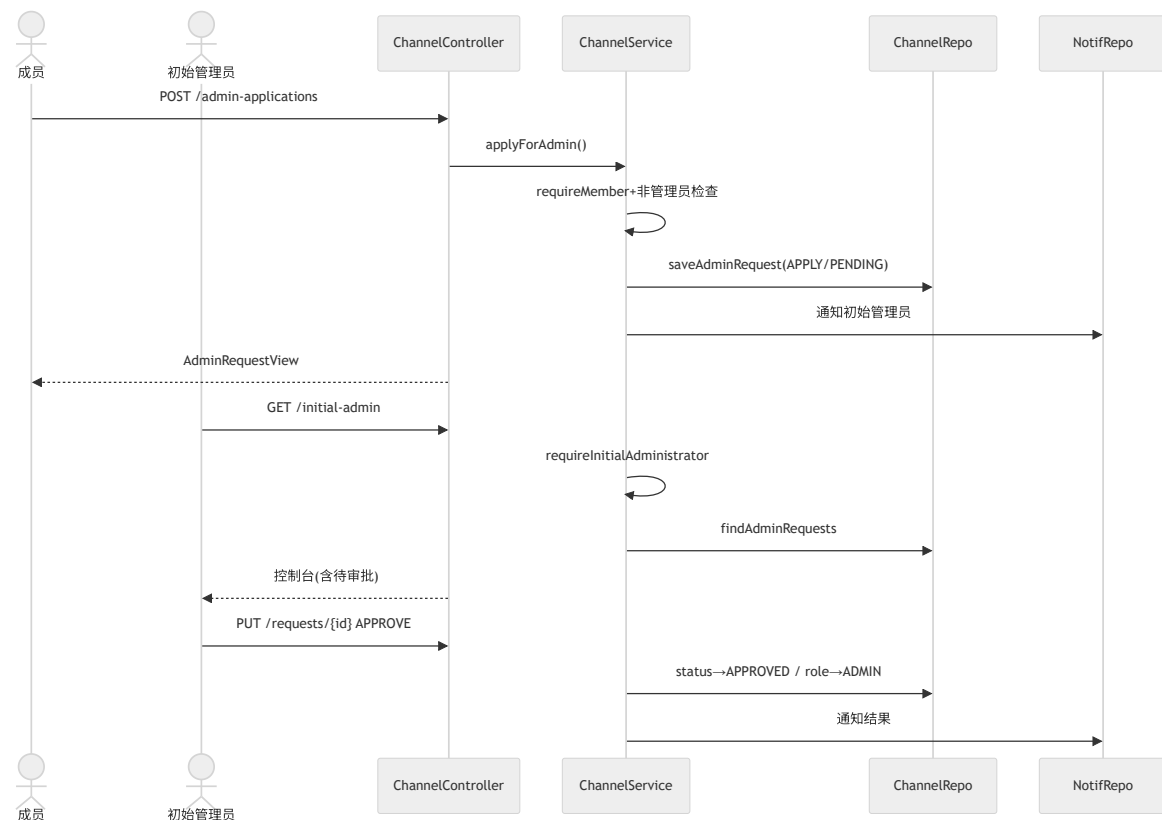
校验链路：邮箱域名 → 用户存在 → BCrypt 密码 → 封禁检查 → 签发 token。被封禁用户即使持有有效 token 也被拦截器拒绝。

3.2 发布笔记（含图片上传）



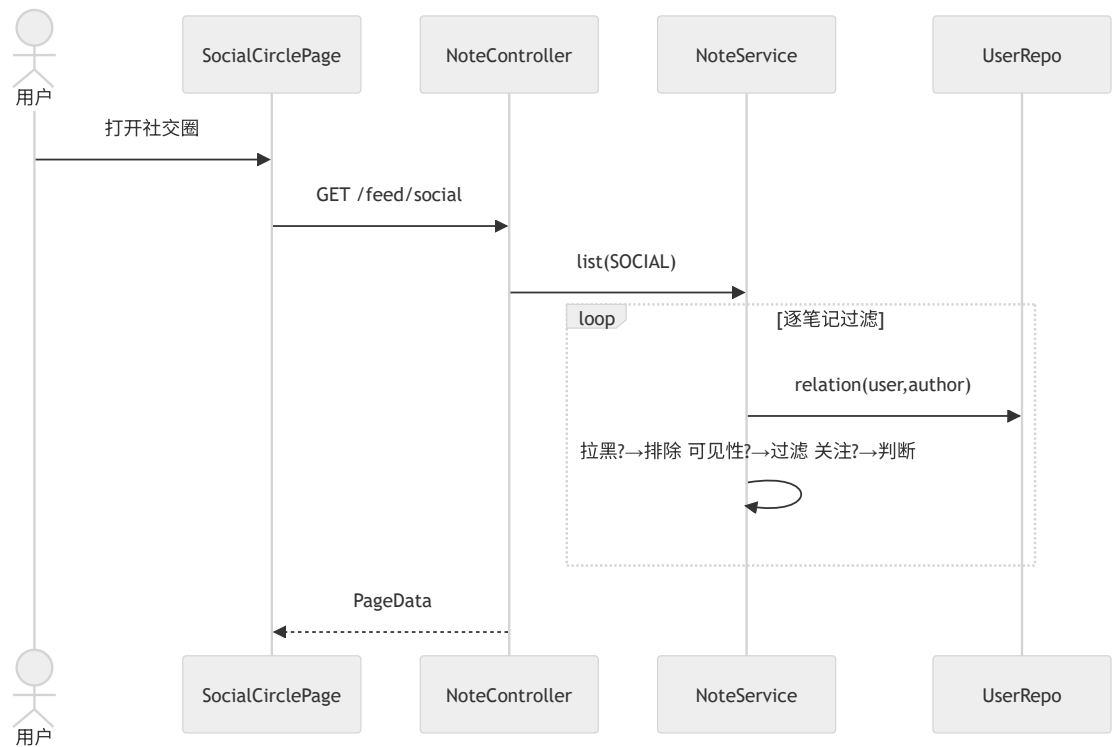
图片与笔记两步提交；数据库仅存 URL 和排序信息，不存图片二进制。本地存 `uploads/images`，联调走 MinIO。

3.3 频道管理员申请与审批



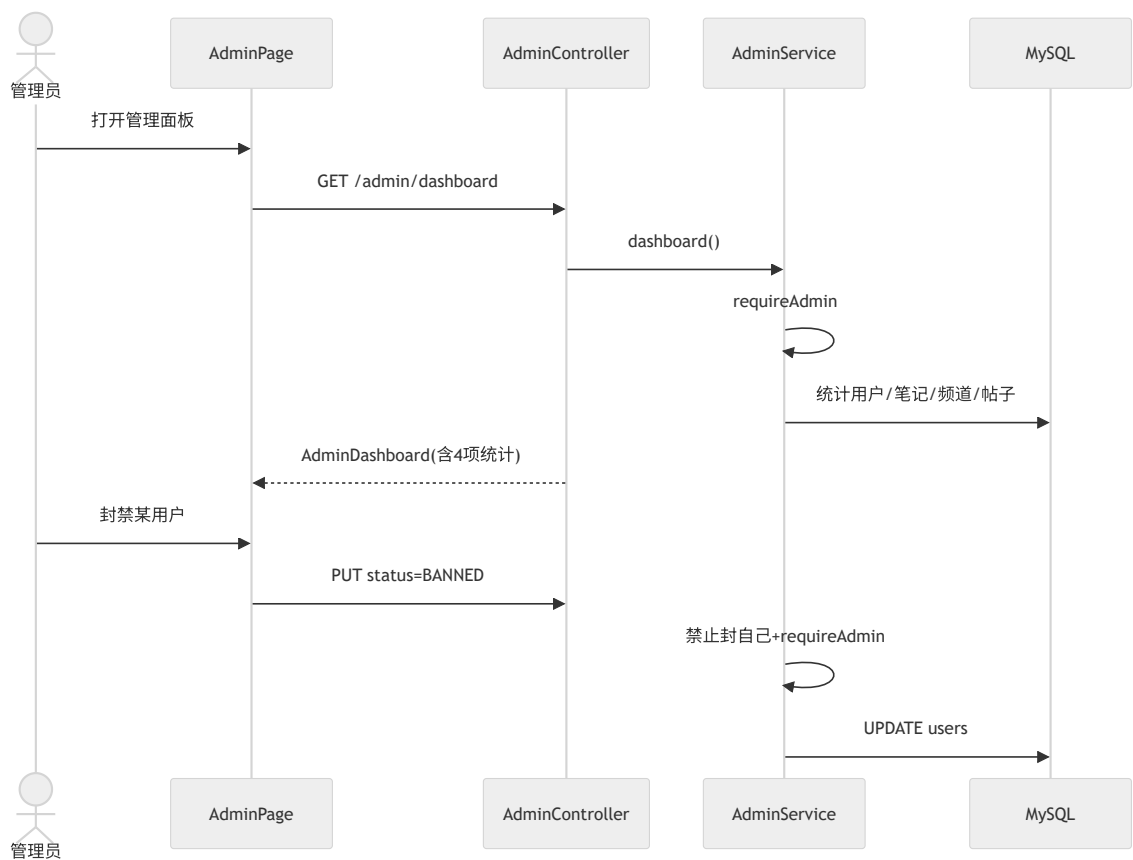
邀请（INVITE）方向同理：初始管理员邀请 → 被邀请者 ACCEPT/DECLINE。仅初始管理员拥有邀请和审批权。

3.4 社交圈笔记流



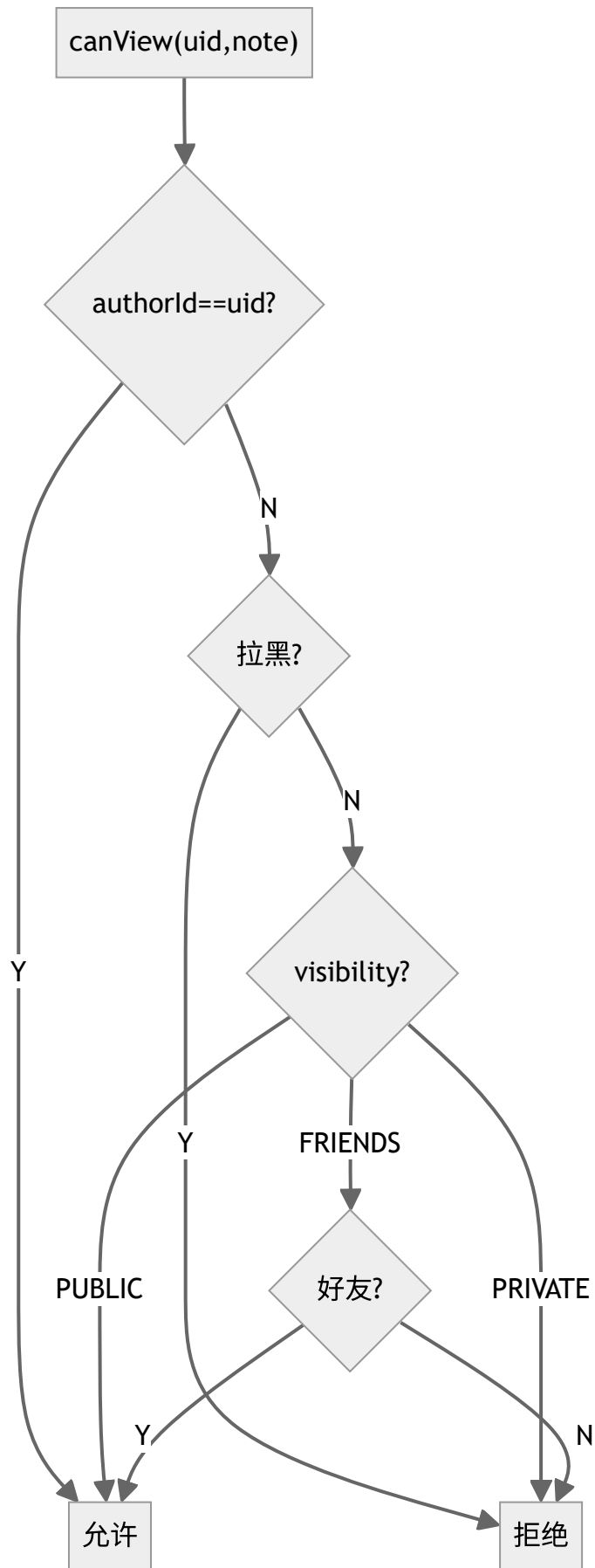
规则：拉黑双边排除；PUBLIC 仅展示关注者/好友的笔记；FRIENDS 仅互关好友可见；自己的笔记不出现。

3.5 全站管理员治理



4. 复杂功能算法设计

4.1 笔记可见性判断



```
function canView(userId, note):  
    if note.authorId == userId: return true  
    if isBlockedEitherWay(userId, note.authorId): return false  
    if note.visibility == PUBLIC: return true  
    if note.visibility == FRIENDS:
```

```
    return relation(userId, note.authorId) == FRIEND
    return false
```

4.2 社交圈可见性判断

在 `canView` 基础上增加关注关系限定，用于社交圈笔记流：

```
function isSocialVisible(userId, note):
    if note.authorId == userId: return false
    if !canView(userId, note): return false
    if note.visibility == FRIENDS: return true
    if note.visibility == PUBLIC:
        rel = relation(userId, note.authorId)
        return rel == FOLLOWING or rel == FRIEND
    return false
```

4.3 个性化推荐评分算法

对笔记、用户、频道三类目标独立评分，首页混合各取 4 条按评分降序展示。评分受隐私开关控制。

笔记推荐评分（多因子加权）：

```
function scoreNote(currentUser, note):
    score = 35
    if sameCollege: score += 18
    if sameGrade: score += 8
    if relation == FRIEND: score += 22
    if relation == FOLLOWING: score += 15
    overlap = countIntersection(note.tags, interestTags)
    score += overlap * 8
    score += min(likeCount*0.7 + commentCount*0.9, 18)
    return score
```

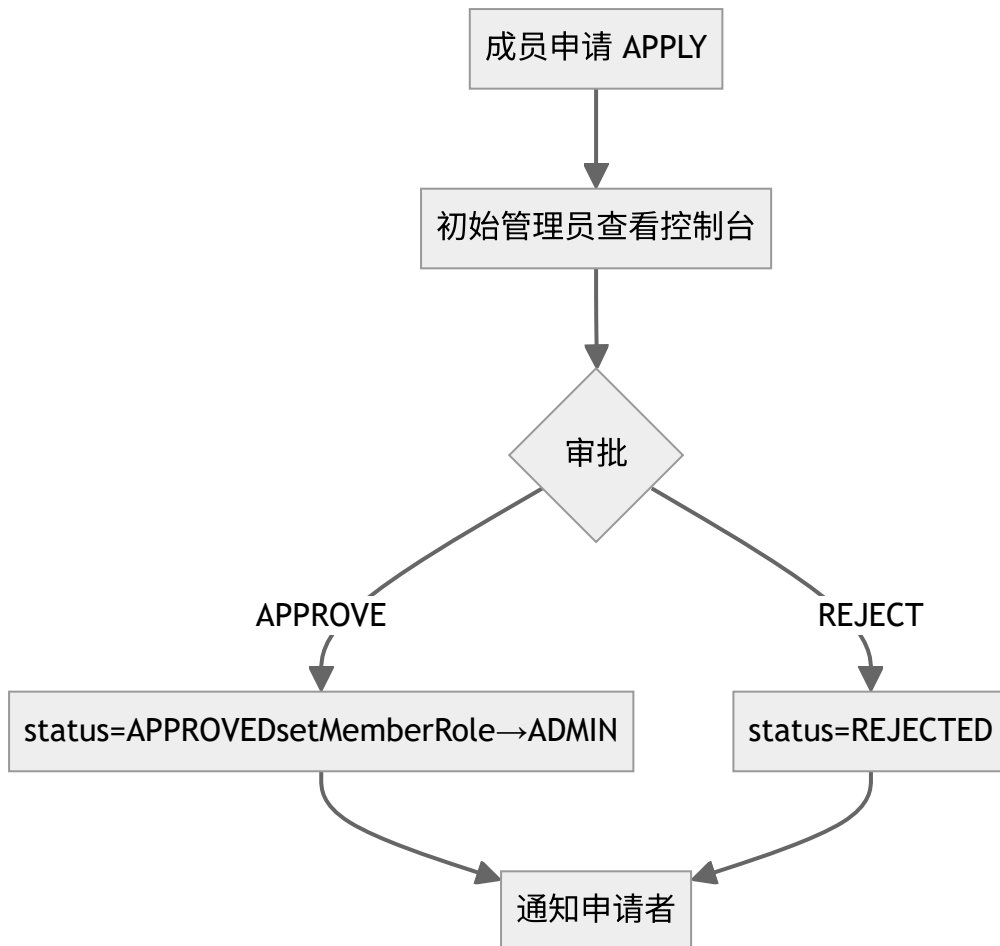
用户推荐评分：基础 30 + 同学院 20 + 同年级 12 + 对方已关注你 14 + $\min(\text{粉丝} \times 0.4 + \text{关注} \times 0.2, 10)$ 。关个性化推荐时不启用学院/年级维度。

频道推荐评分：基础 28 + $\min(\text{成员数} \times 0.25, 18)$ + 公开频道 10 + 好友关系密度 $\times 14$ 。关闭个性化推荐时仅保留公开/成员数维度。

4.4 推荐反馈流程



4.5 频道管理员审批流程



邀请流程：初始管理员发起 INVITE → 被邀请者 ACCEPT/DECLINE。仅初始管理员有邀请和审批权。

4.6 关系状态计算

```
function relation(uid, tid):  
    if isBlockedEitherWay(uid, tid): return BLOCKED  
    i = follows.exists(uid, tid)  
    m = follows.exists(tid, uid)  
    if i && m: return FRIEND  
    if i: return FOLLOWING  
    if m: return FOLLOWER  
    return NONE
```

4.7 通知生成策略

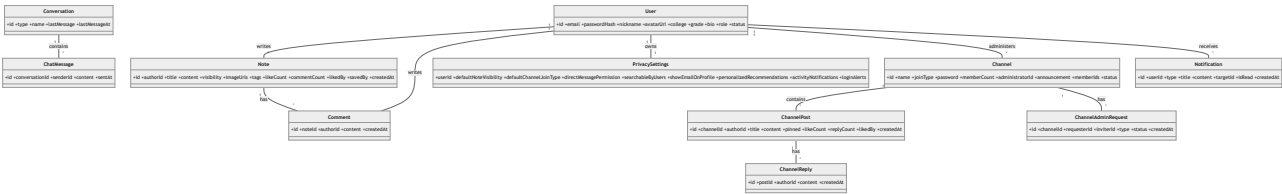
触发事件	接收者	类型
被关注	被关注者	FOLLOW
笔记被评论/点赞	笔记作者	COMMENT / LIKE
频道帖被回复	帖作者	POST_REPLY
管理员申请/邀请	初始管理员/被邀请者	CHANNEL_ADMIN_APPLICATION / INVITE
申请/邀请结果	申请者/邀请者	CHANNEL_ADMIN_RESULT

存在拉黑关系时跳过通知创建。

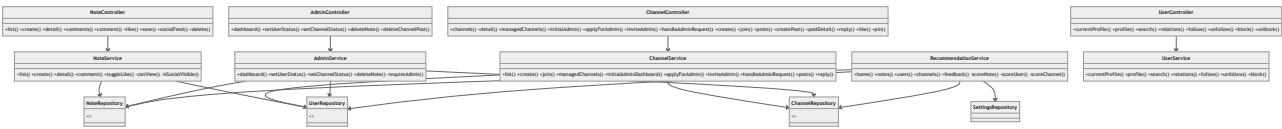
5. 面向对象类图设计

5.1 领域模型

所有领域对象均为 Java record（不可变），通过 Repository 接口持久化。



5.2 后端分层类图



6. 接口设计

前缀 `/api/v1`，统一返回 `{code, message, data}`，认证 `Authorization: Bearer <token>`。`code=0` 成功，非 0 由前端 `client.js` 统一转为 `ApiError`。

模块	方法	路径	说明
认证	POST	/auth/send-code, /register, /login, /reset-password	验证码、注册、登录、找回密码
认证	GET/POST	/auth/me, /auth/logout	当前用户、退出
笔记	GET/POST	/notes	列表(scope/关键字/标签/分页)、发布
笔记	GET/DELETE	/notes/{id}	详情、删除
笔记	GET/POST/DELETE	/notes/{id}/comments	评论列表、发表、删除
笔记	POST	/notes/{id}/like, /save	点赞切换、收藏切换
笔记	GET	/feed/social, /tags	社交圈笔记流、标签列表
用户	GET/PUT	/users/me/profile	查看/编辑个人资料(含统计)
用户	GET	/users/{id}, /search, /relations, /blocks	主页、搜索、关系表、黑名单
用户	POST/DELETE	/users/{id}/follow, /block	关注/取消、拉黑/取消
频道	GET/POST	/channels	列表(joined/关键字/分页)、创建
频道	GET/POST	/channels/{id}, /join	详情、加入(支持密码)
频道	GET/PUT	/channels/managed, /{id}/announcement	管理列表、修改公告
频道	GET/POST	/channels/{id}/initial-admin, /admin-applications, /admin-invitations	管理员控制台、申请、邀请

频道	PUT	/channel-admin-requests/{id}	审批/处理申请邀请
频道	GET/POST	/channels/{id}/posts, /channel-posts/{id}	帖子列表/发布、详情
频道	POST/PUT	/channel-posts/{id}/replies, /like, /pin	回复、点赞、置顶
聊天	GET/POST	/conversations	会话列表、创建(PRI/GROUP)
聊天	GET/POST/PUT	/conversations/{id}/messages, /read	消息列表、发送、已读
通知	GET/PUT	/notifications, /count, /{id}/read, /read-all	列表、未读数、已读
设置	GET/PUT	/settings/privacy	8项隐私开关
推荐	GET	/recommendations/home, /notes, /users, /channels	推荐内容(分页)
推荐	POST	/recommendations/feedback	推荐反馈
举报	POST	/reports	提交 (NOTE/CHANNEL_POST/MESSAGE/USER)
文件	POST	/files/images	上传图片 multipart ≤5MB
管理	GET	/admin/dashboard	管理面板(统计+全量数据)
管理	PUT/DELETE	/admin/users channels/{id}/status, /admin/notes channel-posts/{id}	封禁/解封、删除

错误码：0 成功 / 40000 参数错 / 40001 邮箱密码错 / 40002 验证码错 / 40100 未登录 / 40300 无权限 / 40301 非好友 / 40302 未加入频道 / 40303 频道密码错 / 40400 不存在 / 40900 冲突 / 50000 服务器错误。

7. 数据库物理设计

7.1 基本配置

库名	whu_circle	字符集	utf8mb4	引擎	InnoDB	主键	BIGINT自增/复合	时间	DATETIME(3)
----	------------	-----	---------	----	--------	----	-------------	----	-------------

7.2 数据表清单（25 张）

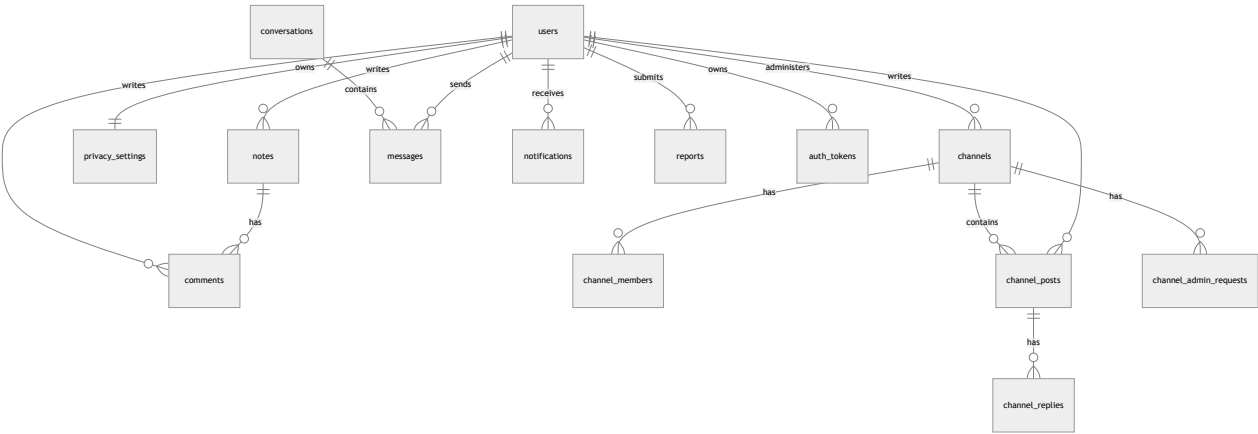
表名	作用
users	用户账号与资料
email_verification_codes	邮箱验证码
user_follows	关注关系
user_blocks	拉黑关系
privacy_settings	隐私设置(8项)
notes	笔记
note_images	笔记图片URL
note_tags	笔记标签
comments	笔记评论

note_likes	笔记点赞
note_saves	笔记收藏
channels	频道
channel_members	频道成员(含role)

表名	作用
channel_admin_requests	管理员申请/邀请
channel_posts	频道帖子
channel_replies	频道回复
channel_post_likes	帖子点赞

conversations	聊天会话	notifications	通知
conversation_members	会话成员	reports	举报记录
messages	聊天消息	auth_tokens	登录凭证
message_read_status	已读状态	recommendation_feedback	推荐反馈

7.3 ER 图

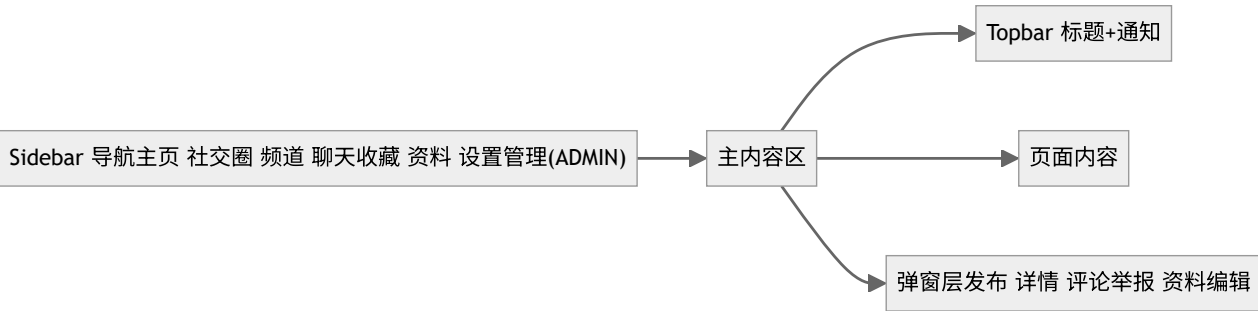


7.4 关键索引

表	索引	目的
notes	idx_note_author_created / idx_note_visibility_created	作者/公开流排序
comments	idx_comment_note_created	评论时间序
channel_admin_requests	idx_car_channel_status / idx_car_requester_status	待审批查询
channel_posts	idx_channel_post_order(channel_id,pinned,created_at)	置顶优先+时间序
notifications	idx_notification_user_read_created	未读通知查询
auth_tokens	idx_auth_token_user / idx_auth_token_expires	登录态/过期清理

8. UI 界面设计

8.1 整体布局



8.2 页面清单

页面	组件	核心功能
登录/注册	AuthPage	登录、注册、找回密码、验证码
主页	HomePage + NotesFeed	笔记流、搜索(笔记/用户)、标签筛选、排序、推荐
社交圈	SocialCirclePage	关注者笔记流、关注/拉黑/私信入口

频道	ChannelsPage	列表、详情、发帖、回复、公告、管理员控制台
聊天	ChatsPage	会话列表、消息窗口、发送、已读标记
收藏	SavedPage	已收藏笔记列表
个人资料	ProfilePage	信息展示与编辑
设置	SettingsPage	8项隐私(可见/搜索/推荐/通知)、黑名单、主题
全站管理	AdminPage	统计面板(用户/频道/笔记/帖子)、封禁/解封、删除

8.3 关键交互

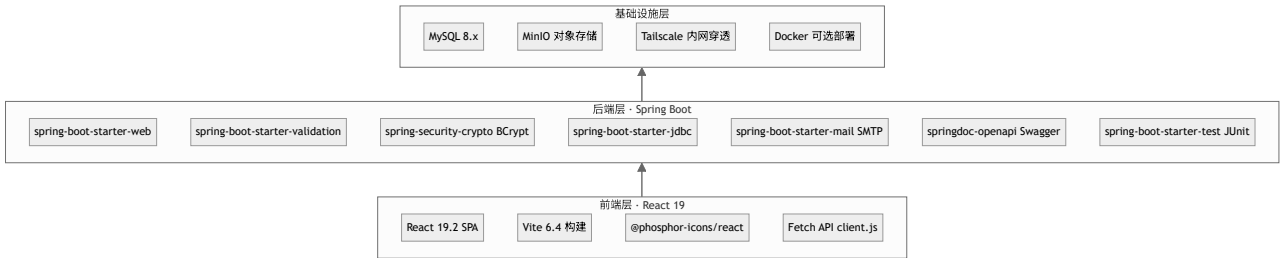
交互	实现
登录态	localStorage["whu-token"], 刷新调 /auth/me 恢复
错误处理	client.js 非0code→ApiError, 页面按场景展示
通知	Topbar通知面板, 管理员邀请可快捷接受/拒绝
频道控制	未加入预览5条帖子, 加入后全功能
管理入口	role==ADMIN 时显示"全站管理"
图片上传	先 POST /files/images 拿 URL, 再随笔记提交
用户搜索	笔记/用户模式切换, 展示昵称学院年级邮箱

8.4 视觉风格

卡片+列表+弹窗+轻量按钮；图标 `@phosphor-icons/react`；CSS class 主题切换；管理界面与普通界面统一视觉体系。

9. 技术栈全景

以下为系统各层所采用的核心技术与组件，按自下而上的部署层次排列。



层次	技术	版本/说明
前端框架	React	19.2.0, 函数组件 + Hooks
构建工具	Vite	6.4.2, HMR 热更新
图标	@phosphor-icons/react	2.1.10
后端框架	Spring Boot	嵌入式 Tomcat
安全	BCrypt + Bearer Token	密码哈希, 无状态认证
数据库	MySQL	8.x, InnoDB, utf8mb4
对象存储	MinIO	S3 兼容, bucket: whu-circle
内网穿透	Tailscale	私有网络, 100.x.y.z
容器化	Docker	docker-compose.yml
API 文档	Swagger / OpenAPI	springdoc-openapi
测试	JUnit	ApiIntegrationTest 等

构建	Maven + npm	前后端独立构建
----	-------------	---------

附录：安全设计要点

层次	措施
认证	Bearer Token, AuthenticationInterceptor 拦截
密码	BCrypt 哈希, 不存明文
验证码	code_hash 存储, 区分 scene, 限尝试次数
注册	校验 @whu.edu.cn 邮箱域名
可见性	PUBLIC/FRIENDS/PRIVATE 三级, 拉黑阻断
管理权	Controller+Service 双重 requireAdmin
频道管理	初始管理员独占邀请/审批
隐私	8项开关: 搜索/邮箱/推荐/通知/登录提醒
文件	jpg/png/gif/webp, ≤5MB, UUID, MinIO密钥不入Git
封禁	持有效token也被拦截器拒绝